

Week 7 In-Class Exercises (Extra)

Part I: Computational Efficiency

Q1: Bubble Sort [***]

Bubble Sort is a simple sorting algorithm which works by repeatedly passing through the list to be sorted, comparing each pair of adjacent items and swapping them if they are in the wrong order. The pass through the list is repeated until no swaps are needed, which means the list is sorted. Below is how Bubble Sort works:

Initial pass:

- Traverse the list from the beginning to the end.
- Compare each pair of adjacent elements.
- Swap them if they are in the wrong order (i.e., the first element is greater than the second for ascending order).
- After the first pass, the largest element will have "bubbled" to the end of the list.

Subsequent passes:

- Repeat the process for the remaining elements (excluding the last sorted elements).
- With each pass, the next largest element will bubble up to its correct position.

Termination:

- The algorithm stops when a pass is completed without any swaps, indicating that the list is sorted.

Example: consider the list [64, 34, 25, 12, 22, 11, 90]

First pass:

- Compare 64 and 34, swap them: [34, 64, 25, 12, 22, 11, 90]
- Compare 64 and 25, swap them: [34, 25, 64, 12, 22, 11, 90]
- Compare 64 and 12, swap them: [34, 25, 12, 64, 22, 11, 90]
- Compare 64 and 22, swap them: [34, 25, 12, 22, 64, 11, 90]
- Compare 64 and 11, swap them: [34, 25, 12, 22, 11, 64, 90]
- Compare 64 and 90, no swap needed: [34, 25, 12, 22, 11, 64, 90]
- Largest element 90 is now in its correct position.

Subsequent passes:

- Repeat the process for the remaining elements, i.e., 34 and 25, and so on until the list is sorted.

Python implementation:

```
1. def bubble_sort(lst):
2.     n = len(lst)
3.     for i in range(n):
4.         # Last i elements are already sorted
5.         for j in range(0, n-i-1):
6.             # Swap the elements if needed
7.             if lst[j] > lst[j+1]:
8.                 temp = lst[j]
9.                 lst[j] = lst[j+1]
10.                lst[j+1] = temp
11.     return lst
```

```
# Example usage
```

```
lst = [64, 34, 25, 12, 22, 11, 90]
```

```
sorted_lst = bubble_sort(lst)
```

```
print("Sorted list:", sorted_lst)
```

Provide a worst-case complexity analysis of the Bubble Sort algorithm. You only need to look at line 3 to line 10 of the above code.

Q2: Smallest Difference [***]

Implement and analyze the worst-case complexity of a function called `find_smallest_diff` that takes in a list `my_list` of `n` integers. The function returns the smallest difference (≥ 0) between any two integers in `my_list`.

For example:

- `find_smallest_diff([1, 2, 3, 4])` returns 1.
- `find_smallest_diff([1, 3, 1, 4])` returns 0.
- `find_smallest_diff([4, 7, 1, 9, 33, 77, 55, 44, 22, 49, 88])` returns 2.

Q3: Find Students [***]

Implement and analyze the worst-case complexity of a function called `find_students` that takes in a list `names` of tuples. Each tuple stores two strings, the first representing a student's first name and the second representing the student's last name. The function returns a list of tuples from `names` where each tuple represents a student with a different last name. If a last name appears more than once in the `names` list, only the student with the first occurrence of that last name should be included in the returned list.

For example:

- `find_students([("James", "Wong"), ("Lily", "Khoo"), ("Peter", "Wong"), ("George", "Lim")])` returns `[("James", "Wong"), ("Lily", "Khoo"), ("George", "Lim")]`. Note there are two students with the last name "Wong" in the list, but only the student appearing first in the list is included in the result.
- `find_students([("James", "Wong"), ("Lily", "Wong"), ("Peter", "Wong"), ("George", "Lim")])` returns `[('James', 'Wong'), ('George', 'Lim')]`.

You can assume that `list.append()` is an $O(1)$ operation.

Q4: Largest Numbers [***]

Implement a function that takes in a list **my_list** of **n** numbers (where **n** is larger than 10) and returns a list of the 10 largest numbers in **my_list**. Ensure that the complexity of your implementation in the worst-case scenario is **O(n)**. Do not use other Python libraries or functions except those mentioned in the hint below.

Hint: you can make use of Python's built-in **sorted**, or **sort** method. Do some research on the complexity of Python's sorting implementation. Note that except Bubble Sort, other sorting methods and their complexity are out of scope and will not be tested.

Q5: Exponential Complexity [***/Out of scope]

Write code for users (e.g., your classmate) to enter a PIN number of **N** digits and attempt to crack it by writing more code to try all possible combinations of the digits. Analyze the worst-case complexity of your code.

To ensure that you can't see the PIN while your user is entering it, you can use `getpass()` – a function which hides the user input:

```
import getpass
PIN = getpass.getpass()
```

Note: exponential complexity analysis is out of scope and will not be tested.

Part II: Review

Q1: Encode Strings [***]

Write a function called `encode_message()` that takes in a string as its only parameter. Call this parameter `text`. You can assume that `text` doesn't contain any space. The function **returns** a string that encodes `text` in the following way: for each segment of consecutive characters that are the same in `text`, the segment will be encoded as the character followed by the number of times it appears in the segment and a space. For example, 'aaa' is encoded as 'a3 ', 't' is encoded as 't1 '. The entire string `text` is encoded by encoding all its segments.

For example,

- `encode_message('aaabbcccccode')` returns 'a3 b2 c5 d1 e1'.
- `encode_message('112333&&$9999999999')` returns '12 21 33 &2 \$1 910'.
- `encode_message('')` returns ''.

Q2: Print Patterns

- a) [***] Define a function called `print_diamond()`. The function takes in an integer `n` as its parameter. You can assume that `n` is at least 2. The function prints a diamond shape where each side of the diamond has exactly `n` '*'.

For example,

- `print_diamond(4)` prints

```

      *
     * *
    *   *
   *     *
  *       *
 *         *
*         *
 *         *
  *       *
   *     *
    *   *
     * *
      *

```

- `print_diamond(2)` prints

```

  *
 * *
 *

```

- b) [***] Define a function called `print_diamond_using_str()`. The function takes in a string `text` as its parameter. The function prints a diamond shape using characters from `text` sequentially, starting from the top of the diamond.

You can assume that

- `text` doesn't contain any space,
- `text` has at least 4 characters,
- the length of `text` is a multiple of 4.

For example,

- `print_diamond('12345678')` prints

```

  1
 2 8
3   7
4 6
 5

```

- `print_diamond('opqrstuvwxyz')` prints

```

  o
 p z
q   y
r     x
 s   w
  t v
   u

```

- c) [***] Define a function called `print_squares()`. The function takes in an integer `n` as its parameter. You can assume that `n` is at least 3 and $(n-3)$ is always a multiple of 4. The function prints out the following kind of pattern, which always consists of $\frac{n+1}{4}$ squares.

For example,

- `print_squares(3)` prints

```
***
* *
***
```

- `print_squares(7)` prints

```
*****
*       *
*   ***   *
* * * * *
*   ***   *
*       *
*****
```

- `print_squares(11)` prints

```
*****
*           *
*   *****   *
* *           * *
* *   ***   * *
* * * * *   * *
* *   ***   * *
* *           * *
*   *****   *
*           *
*****
```